A Unified Closure Framework for Euler Potentials in Resistive MHD: Correct Cartesian Theory, Complete Cylindrical Extension, and the Impossibility of Analytic Spherical Closures

Steven Reid
Independent Researcher
ORCID: 0009-0003-9132-3410
sreid1118@gmail.com

December 2025

Abstract

Euler potentials α and β provide a divergence-free representation of the magnetic field $B = \nabla\alpha \times \nabla\beta$. In ideal MHD, they evolve cleanly along fluid trajectories, but in resistive MHD the naive diffusion equations $\partial_t\alpha = \eta\nabla^2\alpha$, $\partial_t\beta = \eta\nabla^2\beta$ fail to reproduce the correct resistive evolution of B because the vector Laplacian $\nabla^2(\nabla\alpha \times \nabla\beta)$ generates mixed second derivatives absent from $\nabla(\nabla^2\alpha) \times \nabla\beta + \nabla\alpha \times \nabla(\nabla^2\beta)$.

This work provides the first complete, verified, and coordinate-correct closure theory for Euler potential evolution in resistive MHD. We prove an exact analytic closure for Cartesian *bilinear shared-coordinate fields* (where $\alpha = cu$, $\beta = cw$ with $c, u, w \in \{x, y, z\}$), derive all six closure results for cylindrical coordinates, and demonstrate—through analytic, geometric, and computational arguments—that spherical coordinates admit no such analytic closures.

We present full symbolic verification, a stable numerical solver with curvature-aware regularization, and a complete comparison across all three coordinate systems. This establishes a unified framework for when Euler potentials can be used in resistive MHD, what forms admit exact analytic closure, and which settings necessarily require numerical PDE solutions.

Contents

1 Introduction **2**

2 Background and Failure of Naive Diffusion **3**

3 Coordinate Assumptions and Global Disclaimer **3**

4 The General Closure Condition **3**

5 Exact Cartesian Closure Theory **3**

6 Complete Cylindrical Closure Theory **4**

7 Spherical Coordinates: No Analytic Closures **5**

8 Verification via Symbolic Computation **5**

9 Stable Numerical Solver **5**

10 Unified Comparison Across Coordinate Systems **6**

11 Discussion and Limitations **6**

12 Conclusion **6**

Appendix A: Cartesian Closure Derivation **8**

Appendix B: Cylindrical Closure Derivations **12**

Appendix C: Spherical Non-Closure Proof **19**

A Appendix D: Python Verification Code **22**

Appendix D: Python Verification Suite **22**

1 Introduction

Magnetohydrodynamics (MHD) describes the motion of conducting fluids under magnetic fields. The magnetic field evolves according to the resistive induction equation:

$$\frac{\partial B}{\partial t} = \nabla \times (v \times B) + \eta \nabla^2 B. \tag{1}$$

Representing the magnetic field using Euler potentials,

$$B = \nabla\alpha \times \nabla\beta, \tag{2}$$

automatically ensures $\nabla \cdot B = 0$. However, the evolution of α and β in the resistive setting is subtle: the naive diffusion system

$$\partial_t \alpha = \eta \nabla^2 \alpha, \qquad \partial_t \beta = \eta \nabla^2 \beta \tag{3}$$

does *not* reproduce $\eta \nabla^2 B$.

To repair this mismatch, we introduce closure source terms

$$\partial_t \alpha = \eta \nabla^2 \alpha + S_\alpha, \qquad \partial_t \beta = \eta \nabla^2 \beta + S_\beta, \tag{4}$$

and require the closure condition

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = R, \tag{5}$$

where R is the remainder term

$$R = \eta \nabla^2 (\nabla\alpha \times \nabla\beta) - \nabla(\eta \nabla^2 \alpha) \times \nabla\beta - \nabla\alpha \times \nabla(\eta \nabla^2 \beta).$$

This paper provides a *complete, coordinate-correct classification* of all solvable cases, verified symbolically and numerically.

2 Background and Failure of Naive Diffusion

Expanding the diffusion of the Euler potentials produces:

$$\partial_t B = \nabla(\eta\nabla^2\alpha) \times \nabla\beta + \nabla\alpha \times \nabla(\eta\nabla^2\beta).$$

But in general

$$\eta\nabla^2(\nabla\alpha \times \nabla\beta) \neq \nabla(\eta\nabla^2\alpha) \times \nabla\beta + \nabla\alpha \times \nabla(\eta\nabla^2\beta).$$

The mismatch is the remainder term R, which contains mixed second derivatives of α and β.

A closure (S_α, S_β) is valid if and only if

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = R.$$

We show that:

- In **Cartesian coordinates**, this is exactly solvable for a nontrivial class of fields (bilinear shared-coordinate fields).
- In **cylindrical coordinates**, a different set of six cases are solvable, fully derived in this work.
- In **spherical coordinates**, no analytic closures exist due to curvature coupling and angular scale factors.

3 Coordinate Assumptions and Global Disclaimer

The original closure formula

$$S_\alpha = \eta\,\frac{u}{c}, \qquad S_\beta = \eta\,\frac{w}{c},$$

is valid *only in Cartesian coordinates.* In curvilinear geometries, metric scale factors h_i (*e.g.*, $h_\theta = r$, $h_\phi = r\sin\theta$) introduce additional derivative couplings that completely alter the closure structure.

This is the central correction of this unified preprint: the Cartesian closure formula does not generalize.

4 The General Closure Condition

Given α and β, the required closure sources must satisfy:

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = R. \tag{6}$$

This is a system of three PDEs (vector equation) for two scalar unknowns S_α, S_β. Generically, it is overdetermined.

Only highly structured α, β admit solutions.

5 Exact Cartesian Closure Theory

We consider "bilinear shared-coordinate" fields:

$$\alpha = cu, \qquad \beta = cw, \qquad c, u, w \in \{x, y, z\}, \quad u \neq w.$$

Theorem 1 (Exact Cartesian Closure)**.** *For any bilinear pair $\alpha = cu$, $\beta = cw$ in Cartesian coordinates, the closure sources*

$$S_\alpha = \eta\,\frac{u}{c}, \qquad S_\beta = \eta\,\frac{w}{c},$$

satisfy the closure condition (6) *exactly.*

The proof (Appendix A) uses the constancy of second derivatives of linear coordinates and direct evaluation of the Hessians.

All six permutations are tabulated:

α	β	shared c	S_α	S_β
xy	xz	x	$\eta y/x$	$\eta z/x$
yz	yx	y	$\eta z/y$	$\eta x/y$
zx	zy	z	$\eta x/z$	$\eta y/z$

6 Complete Cylindrical Closure Theory

Cylindrical coordinates (r, θ, z) introduce scale factors:

$$\nabla f = \left(\partial_r f,\, \frac{1}{r}\partial_\theta f,\, \partial_z f\right),$$

and the vector Laplacian contains curvature terms such as $-B_r/r^2$, $(2/r^2)\partial_\theta B_r$, etc.

The Cartesian pattern fails.

We classify all bilinear forms of the type

$$\alpha = c\,u, \qquad \beta = c\,w, \qquad c, u, w \in \{r, \theta, z\}, \quad u \neq w,$$

and compute which admit closures.

Result: all six cases solved

α	β	shared	R	S_α	S_β
$r\theta$	rz	r	0	0	0
rz	$r\theta$	r	0	0	0
$r\theta$	θz	θ	$\left(\frac{2\eta\theta}{r^2},\, 0,\, \frac{2\eta\theta z}{r^3}\right)$	$\eta\theta/r$	$\eta\theta z/r^2$
θz	$r\theta$	θ	$\left(-\frac{2\eta\theta}{r^2},\, 0,\, -\frac{2\eta\theta z}{r^3}\right)$	$\eta\theta z/r^2$	$\eta\theta/r$
θz	rz	z	$\left(-\frac{4\eta z}{r^2},\, 0,\, -\frac{2\eta(r^2+z^2)}{r^3}\right)$	$\theta(\eta/z - \eta z/r^2)$	$\eta r/z - 3\eta z/r$
rz	θz	z	$\left(\frac{4\eta z}{r^2},\, 0,\, \frac{2\eta(r^2+z^2)}{r^3}\right)$	$\eta r/z - 3\eta z/r$	$\theta(\eta/z - \eta z/r^2)$

Proofs for all cases appear in Appendix B.

Key observations

- Two cases (r-sharing) require no correction: $R = 0$.
- θ-sharing introduces $1/r$ curvature terms.
- z-sharing introduces mixed curvature + axial coupling.
- The symmetry between pairs (3,4) and (5,6) is exact.

7 Spherical Coordinates: No Analytic Closures

Spherical coordinates (r, θ, ϕ) have scale factors:

$$h_r = 1, \qquad h_\theta = r, \qquad h_\phi = r \sin\theta.$$

Even the simplest "bilinear" fields such as $\alpha = r\theta$ or $\alpha = r\phi$ produce gradient components:

$$\nabla(r\theta) = (\theta,\, r^{-1}r,\, 0), \quad \nabla(r\phi) = (\phi,\, 0,\, (r\sin\theta)^{-1}r).$$

The vector Laplacian contains strong angular coupling:

$$(\nabla^2 B)_r = \nabla^2 B_r - 2\frac{B_r}{r^2} - \frac{2}{r^2 \sin\theta}\frac{\partial(B_\theta \sin\theta)}{\partial\theta} - \frac{2}{r^2\sin\theta}\frac{\partial B_\phi}{\partial\phi},$$

etc.

Theorem 2 (Non-existence of Analytic Spherical Bilinear Closures)**.** *No bilinear pair (α, β) in spherical coordinates admits analytic scalar closures (S_α, S_β) satisfying the closure equation. Curvature-induced trigonometric couplings force the system of three PDEs in two unknowns to be overdetermined with no solutions.*

Proof outline in Appendix C.

Remark 1. *This establishes the necessity of numerical closure in spherical geometries.*

8 Verification via Symbolic Computation

We implement exact symbolic verification in SymPy for all Cartesian and cylindrical cases. Given $\alpha, \beta, S_\alpha, S_\beta$, we compute:

$$\text{error} = (\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta) - \left(\eta\nabla^2(\nabla\alpha \times \nabla\beta) - \nabla(\eta\nabla^2\alpha) \times \nabla\beta - \nabla\alpha \times \nabla(\eta\nabla^2\beta)\right).$$

A closure is correct iff all three components of error simplify to zero.
All six cylindrical cases pass exactly.
The full verification script appears in Appendix D.

9 Stable Numerical Solver

We provide a curvature-aware numerical solver for resistive evolution of Euler potentials. Key features include:

- implicit-stable diffusion in Cartesian and cylindrical grids,
- regularization of singularities via $a/b \to ab/(b^2 + \epsilon^2)$,
- safe cylindrical Laplacian with correct radial behavior,
- closure-injection layer using the computed formulas,
- optional fallback to pure vector-MHD evolution.

Full pseudocode and Python implementation appear in Appendix D.

10 Unified Comparison Across Coordinate Systems

Feature	Cartesian	Cylindrical	Spherical
Coordinate scale factors	none	$1/r$	$1/r$, $1/(r\sin\theta)$
Analytic bilinear closures	yes (all)	yes (all 6)	none
Number of solvable cases	6	6	0
Curvature effects	absent	moderate	strong
Need for numerical closure	no	optional	mandatory

11 Discussion and Limitations

- Closure formulas are coordinate-dependent; no general coordinate-free closure exists.
- Analytic closures exist only for bilinear fields; nonlinear or turbulent fields require numerical PDE closure.
- Singularities near $r = 0$ or $\theta = \pi/2$ demand numerical regularization.

12 Conclusion

This work establishes the first complete and verified closure framework for Euler potentials in resistive MHD:

- exact Cartesian closures,
- complete cylindrical closures,
- proof of impossibility in spherical coordinates,
- full numerical and symbolic verification.

Euler potentials remain valuable but only within the sharply defined domain of validity established here.

AI Acknowledgment

The author acknowledges the use of AItools for assistance with clarity editing, symbolic computation checks through wolfram cloud, and code-generation scaffolding during the preparation of this manuscript. All mathematical derivations, theoretical results, numerical implementations, and scientific interpretations were independently developed, verified, and validated by the author. AI tools served solely as auxiliary assistants and did not originate any scientific claims or conclusions in this work.

Reference Note on Prior Work

This research was informed in part by the author's earlier studies on recursive structures and geometric entropy, including:

Steven Reid, *Topological Adam: An Energy-Stabilized Optimizer Inspired by Magnetohydrodynamic Coupling*, Zenodo (2025). DOI: **10.5281/zenodo.17489664**. Available at: `https://github.com/RRG314/topological-adam`.

- Steven Reid, *Recursive Division Tree: A Log-Log Algorithm for Integer Depth*, DOI: **10.5281/zenodo.17487650**.
- Steven Reid, *Recursive Geometric Entropy: A Unified Framework for Information-Theoretic Shape Analysis*, DOI: **10.5281/zenodo.17882309**.

These works helped shape the conceptual motivation for exploring geometric and coordinate-structured closure behavior in resistive Euler-potential magnetohydrodynamics.

References

[1] D. P. Stern, *Euler Potentials*, American Journal of Physics **38**, 494–501 (1970).

[2] T. Sakurai, *Computation of force-free magnetic fields*, Solar Physics **63**, 25–44 (1979).

[3] A. H. Boozer, *Euler potentials and magnetic fields*, Journal of Plasma Physics **35**, 133–139 (1986).

[4] G. Hornig and K. Schindler, *Magnetic reconnection in space plasmas: Euler potentials and obstacles*, Journal of Geophysical Research **101**, 15431–15446 (1996).

[5] D. Price and M. Bate, *Magnetic fields in smoothed particle hydrodynamics: I. Formulation and the divergence constraint*, Journal of Computational Physics **225**, 963–987 (2007).

[6] R. Kulsrud, *Plasma Physics for Astrophysics*, Princeton University Press (2005).

[7] A. Brandenburg and K. Subramanian, *Astrophysical magnetic fields and nonlinear dynamo theory*, Physics Reports **417**, 1–209 (2005).

[8] J. P. Goedbloed, R. Keppens, and S. Poedts, *Advanced Magnetohydrodynamics: With Applications to Laboratory and Astrophysical Plasmas*, Cambridge University Press (2010).

[9] G. Arfken and H. Weber, *Mathematical Methods for Physicists*, 6th ed., Elsevier (2005).

[10] P. Morse and H. Feshbach, *Methods of Theoretical Physics*, McGraw–Hill (1953).

[11] G. Tóth, *The $\nabla \cdot B = 0$ constraint in shock-capturing magnetohydrodynamics codes*, Journal of Computational Physics **161**, 605–652 (2000).

[12] J. M. Stone and M. L. Norman, *ZEUS-2D: A radiation magnetohydrodynamics code for astrophysical flows*, Astrophysical Journal Supplement Series **80**, 753–790 (1992).

[13] A. Dedner et al., *Hyperbolic divergence cleaning for the MHD equations*, Journal of Computational Physics **175**, 645–673 (2002).

[14] Steven Reid, *Recursive Division Tree: A Log-Log Algorithm for Integer Depth*, Zenodo (2025). DOI: **10.5281/zenodo.17487650**.

[15] Steven Reid, *Recursive Geometric Entropy: A Unified Framework for Information-Theoretic Shape Analysis*, Zenodo (2025). DOI: **10.5281/zenodo.17882309**.

Appendix A: Full Cartesian Closure Derivation

In this appendix we provide a complete derivation of the exact closure formula for bilinear Euler potentials in Cartesian coordinates, as stated in the main text:

$$\alpha = c\,u, \qquad \beta = c\,w, \qquad c, u, w \in \{x, y, z\}, \quad u \neq w,$$

with closure sources

$$S_\alpha = \eta\,\frac{u}{c}, \qquad S_\beta = \eta\,\frac{w}{c}.$$

We work strictly in Cartesian coordinates (x, y, z) with the standard Euclidean metric.

A.1. Setup and notation

Let $\alpha(x, y, z)$ and $\beta(x, y, z)$ be two scalar fields. The magnetic field is defined by the Euler-potential representation

$$B = \nabla\alpha \times \nabla\beta. \tag{7}$$

The resistive induction equation in the absence of flow ($v \equiv 0$) reads

$$\partial_t B = \eta\nabla^2 B, \tag{8}$$

where $\eta > 0$ is the (constant) magnetic diffusivity and ∇^2 is the usual Cartesian Laplacian.

If we evolve α and β by diffusion alone,

$$\partial_t\alpha = \eta\nabla^2\alpha, \qquad \partial_t\beta = \eta\nabla^2\beta, \tag{9}$$

then

$$\partial_t B = \nabla(\eta\nabla^2\alpha) \times \nabla\beta + \nabla\alpha \times \nabla(\eta\nabla^2\beta). \tag{10}$$

In general,

$$\eta\nabla^2(\nabla\alpha \times \nabla\beta) \neq \nabla(\eta\nabla^2\alpha) \times \nabla\beta + \nabla\alpha \times \nabla(\eta\nabla^2\beta). \tag{11}$$

Their difference is the remainder field R:

$$R = \eta\nabla^2(\nabla\alpha \times \nabla\beta) - \nabla(\eta\nabla^2\alpha) \times \nabla\beta - \nabla\alpha \times \nabla(\eta\nabla^2\beta). \tag{12}$$

To obtain a valid closure we introduce source terms S_α, S_β in the scalar evolution equations:

$$\partial_t\alpha = \eta\nabla^2\alpha + S_\alpha, \qquad \partial_t\beta = \eta\nabla^2\beta + S_\beta, \tag{13}$$

which yields

$$\partial_t B = \eta\nabla^2 B + \nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta. \tag{14}$$

For this to agree with the resistive induction equation we require

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = R, \tag{15}$$

with R defined by (12). This is a vector equation in $\mathbb{R}^3$, i.e. three scalar PDEs for the two unknown scalar fields S_α, S_β.

We now show that for a special but important class of fields, the system is exactly solvable.

A.2. Concrete example: $\alpha = xy$, $\beta = xz$

We begin with the specific bilinear pair

$$\alpha = xy, \qquad \beta = xz. \tag{16}$$

The gradients are

$$\nabla\alpha = (\partial_x\alpha, \partial_y\alpha, \partial_z\alpha) = (y, x, 0), \tag{17}$$
$$\nabla\beta = (\partial_x\beta, \partial_y\beta, \partial_z\beta) = (z, 0, x). \tag{18}$$

The magnetic field is

$$\begin{aligned} B &= \nabla\alpha \times \nabla\beta && (19)\\ &= (y, x, 0) \times (z, 0, x) && (20)\\ &= \big(x \cdot x - 0 \cdot 0,\ 0 \cdot z - y \cdot x,\ y \cdot 0 - x \cdot z\big) && (21)\\ &= (x^2,\ -xy,\ -xz). && (22) \end{aligned}$$

Step 1: scalar Laplacians of α and β. We compute:

$$\nabla^2\alpha = \partial_x^2(xy) + \partial_y^2(xy) + \partial_z^2(xy) = 0 + 0 + 0 = 0, \tag{23}$$

and similarly

$$\nabla^2\beta = \partial_x^2(xz) + \partial_y^2(xz) + \partial_z^2(xz) = 0 + 0 + 0 = 0. \tag{24}$$

Thus

$$\nabla(\eta\nabla^2\alpha) = 0, \qquad \nabla(\eta\nabla^2\beta) = 0, \tag{25}$$

and the second and third terms in (12) vanish:

$$R = \eta\nabla^2 B. \tag{26}$$

Step 2: vector Laplacian of B. We compute the Laplacian of each component of B:

$$\nabla^2 B_x = \nabla^2(x^2) = \partial_x^2(x^2) + \partial_y^2(x^2) + \partial_z^2(x^2) = 2 + 0 + 0 = 2, \tag{27}$$
$$\nabla^2 B_y = \nabla^2(-xy) = -\partial_x^2(xy) - \partial_y^2(xy) - \partial_z^2(xy) = 0, \tag{28}$$
$$\nabla^2 B_z = \nabla^2(-xz) = -\partial_x^2(xz) - \partial_y^2(xz) - \partial_z^2(xz) = 0. \tag{29}$$

Therefore

$$\nabla^2 B = (2,\ 0,\ 0), \qquad R = \eta\nabla^2 B = (2\eta, 0, 0). \tag{30}$$

Step 3: closure equations. We write

$$\nabla S_\alpha = (\partial_x S_\alpha, \partial_y S_\alpha, \partial_z S_\alpha), \qquad \nabla S_\beta = (\partial_x S_\beta, \partial_y S_\beta, \partial_z S_\beta).$$

The closure condition (15) becomes:

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = (2\eta, 0, 0). \tag{31}$$

A straightforward component-wise expansion using $\nabla\alpha = (y, x, 0)$, $\nabla\beta = (z, 0, x)$ yields the system:

$$\text{x-component:} \quad \partial_y S_\alpha \cdot x + x\,\partial_z S_\beta = 2\eta, \tag{32}$$
$$\text{y-component:} \quad -\partial_x S_\alpha \cdot x - y\,\partial_z S_\beta = 0, \tag{33}$$
$$\text{z-component:} \quad -\partial_y S_\alpha \cdot z + y\,\partial_y S_\beta - x\,\partial_x S_\beta = 0. \tag{34}$$

These are three first-order PDEs for the two unknowns S_α, S_β.

Step 4: solving the system. We seek a solution of the form

$$S_\alpha = \eta\,\frac{y}{x}, \qquad S_\beta = \eta\,\frac{z}{x}, \tag{35}$$

motivated by dimensional and symmetry considerations (and later justified by a general argument). We compute

$$\partial_x S_\alpha = \partial_x(\eta y/x) = -\eta y/x^2, \quad \partial_y S_\alpha = \partial_y(\eta y/x) = \eta/x, \quad \partial_z S_\alpha = 0, \tag{36}$$

$$\partial_x S_\beta = \partial_x(\eta z/x) = -\eta z/x^2, \quad \partial_y S_\beta = 0, \quad \partial_z S_\beta = \partial_z(\eta z/x) = \eta/x. \tag{37}$$

We now check each equation:

Equation (32)**.**

$$\partial_y S_\alpha \cdot x + x\,\partial_z S_\beta = \left(\frac{\eta}{x}\right) x + x\left(\frac{\eta}{x}\right) = \eta + \eta = 2\eta,$$

as required.

Equation (33)**.**

$$-\partial_x S_\alpha \cdot x - y\,\partial_z S_\beta = -\left(-\frac{\eta y}{x^2}\right) x - y\left(\frac{\eta}{x}\right) = \frac{\eta y}{x} - \frac{\eta y}{x} = 0,$$

as required.

Equation (34)**.**

$$-\partial_y S_\alpha \cdot z + y\,\partial_y S_\beta - x\,\partial_x S_\beta = -\left(\frac{\eta}{x}\right) z + y \cdot 0 - x\left(-\frac{\eta z}{x^2}\right) = -\frac{\eta z}{x} + \frac{\eta z}{x} = 0.$$

All three equations are satisfied, and hence $S_\alpha = \eta y/x$, $S_\beta = \eta z/x$ provide an exact closure for the pair $\alpha = xy$, $\beta = xz$.

Equivalently, we may check directly that

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = (2\eta, 0, 0) = R, \tag{38}$$

recovering the result from the main text.

A.3. Generalization to $\alpha = c\,u$, $\beta = c\,w$

We now generalize the above computation to an arbitrary bilinear pair

$$\alpha = c\,u, \qquad \beta = c\,w,$$

where c, u, w are chosen from $\{x, y, z\}$ with $u \neq w$. Without loss of generality, we may view (c, u, w) as some permutation of (x, y, z), and work in the same Cartesian coordinate system. The key observation is that the coordinate functions x, y, z are linear, so all second derivatives of α and β are constants (or zero), and the structure of R is invariant under such permutations.

Gradients. For a given triple (c, u, w) we have

$$\alpha = c\,u, \qquad \beta = c\,w. \tag{39}$$

The gradient of α is

$$\nabla\alpha = u\,\nabla c + c\,\nabla u, \tag{40}$$

and similarly

$$\nabla\beta = w\,\nabla c + c\,\nabla w. \tag{41}$$

Since c, u, w are coordinate functions (*e.g.*, x or y or z), their gradients are constant unit basis vectors in the corresponding coordinate directions, and all mixed partial derivatives are constants (or zero).

Scalar Laplacians. Each of c, u, w is linear, hence

$$\nabla^2 c = \nabla^2 u = \nabla^2 w = 0.$$

Therefore

$$\nabla^2 \alpha = \nabla^2 (cu) = 0, \qquad \nabla^2 \beta = \nabla^2 (cw) = 0,$$

and as before

$$R = \eta \nabla^2 B, \tag{42}$$

with

$$B = \nabla \alpha \times \nabla \beta.$$

Structure of B and R. The field B is a quadratic polynomial in (x, y, z), because α and β are bilinear and gradients reduce each to linear degree. The Laplacian of a quadratic polynomial is constant:

$$\nabla^2 B = (\text{constant vector depending on } c, u, w).$$

In the example $(c, u, w) = (x, y, z)$ we obtained

$$\nabla^2 B = (2, 0, 0).$$

If we cyclically permute (x, y, z), the roles of the components of B and of $\nabla^2 B$ permute accordingly. Thus for each bilinear triple (c, u, w) we have

$$R = \eta \nabla^2 B = (0, 0, 0) \text{ except along the direction of } c, \tag{43}$$

i.e. R is a constant vector pointing along the shared coordinate c, with some constant magnitude depending only on the choice of (c, u, w). Concretely, if (c, u, w) is a permutation of (x, y, z) with c in the x-position, then

$$R = (2\eta, 0, 0);$$

if c is in the y-position, then $R = (0, 2\eta, 0)$; and so on.

Ansatz for closure. Guided by the explicit solution in the (x, y, z) example, and by the fact that R is constant and aligned with the shared coordinate c, we make the ansatz

$$S_\alpha = \eta \, \frac{u}{c}, \qquad S_\beta = \eta \, \frac{w}{c}. \tag{44}$$

More precisely, we interpret u/c and w/c as coordinate ratios in the same Cartesian system, i.e. $u/c = y/x$, z/x, x/y, etc., depending on the choice of (c, u, w). This ensures that S_α and S_β are homogeneous of degree zero (dimensionless), consistent with the fact that R is constant.

We then compute

$$\nabla S_\alpha, \quad \nabla S_\beta$$

explicitly and evaluate

$$\nabla S_\alpha \times \nabla \beta + \nabla \alpha \times \nabla S_\beta.$$

Because the coordinate triple (c, u, w) is obtained from (x, y, z) by a permutation, and because the gradient and Laplacian operators are invariant under such permutations, the algebra of the general case is identical (up to relabelling) to the concrete computation carried out for $(\alpha, \beta) = (xy, xz)$. That is:

- The PDE system corresponding to the closure condition (15) has the same structural form as (32)–(34), with (x, y, z) simply replaced by the permuted (c, u, w).

- The proposed closure
$$S_\alpha = \eta\,\frac{u}{c}, \qquad S_\beta = \eta\,\frac{w}{c}$$
satisfies this system identically, by the same cancellation mechanism demonstrated in the worked example.

- The resulting closure field
$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta$$
is equal to $R = \eta\nabla^2 B$ and points along the c-direction, with the correct constant magnitude.

Formally, one can phrase this as an invariance argument under the action of the permutation group S_3 on the coordinate axes. Since the differential operators and the bilinear structure of α, β are equivariant under such permutations, and since the closure has been verified explicitly for one representative $(c, u, w) = (x, y, z)$, it holds for all permutations.

We thus obtain:

Theorem 3 (Cartesian bilinear closure)**.** *Let (x, y, z) be standard Cartesian coordinates on $\mathbb{R}^3$ and let c, u, w be a permutation of (x, y, z) with $u \neq w$. Consider Euler potentials*
$$\alpha = c\,u, \qquad \beta = c\,w,$$
and define the magnetic field
$$B = \nabla\alpha \times \nabla\beta.$$
Then the resistive remainder R defined by (12) is a constant vector aligned with c, and the scalar fields
$$S_\alpha = \eta\,\frac{u}{c}, \qquad S_\beta = \eta\,\frac{w}{c}$$
satisfy the closure equation
$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = R$$
for all (x, y, z) with $c \neq 0$. In particular, evolving α and β via
$$\partial_t\alpha = \eta\nabla^2\alpha + S_\alpha, \qquad \partial_t\beta = \eta\nabla^2\beta + S_\beta,$$
ensures that B satisfies the resistive induction equation
$$\partial_t B = \eta\nabla^2 B$$
exactly.

Remark 2. *The apparent singularity of S_α and S_β at the hypersurface $\{c = 0\}$ is purely a coordinate effect: the bilinear fields themselves (and the magnetic field B) can be restricted to half-spaces where c does not change sign, so that the closure remains regular on the domain of interest. In numerical implementations, one may further regularize the ratio u/c using soft division, as described in Appendix D.*

Appendix B: Complete Cylindrical Closure Derivations

In this appendix we derive the closure sources for all six bilinear shared-coordinate Euler potential fields in cylindrical coordinates. We work in (r, θ, z) with the usual ranges:
$$r > 0, \qquad 0 \leq \theta < 2\pi, \qquad z \in \mathbb{R}.$$

B.1. Cylindrical operators and notation

Let $(\hat{e}_r, \hat{e}_\theta, \hat{e}_z)$ denote the standard orthonormal cylindrical basis. For a scalar field $f(r, \theta, z)$:

$$\nabla f = \hat{e}_r\, \partial_r f + \hat{e}_\theta\, \frac{1}{r}\partial_\theta f + \hat{e}_z\, \partial_z f. \tag{45}$$

The scalar Laplacian in cylindrical coordinates is

$$\nabla^2 f = \frac{1}{r}\, \partial_r(r\partial_r f) + \frac{1}{r^2}\, \partial_\theta^2 f + \partial_z^2 f. \tag{46}$$

If $B = B_r\hat{e}_r + B_\theta\hat{e}_\theta + B_z\hat{e}_z$, then the vector Laplacian (for orthonormal components) is

$$(\nabla^2 B)_r = \nabla^2 B_r - \frac{B_r}{r^2} - \frac{2}{r^2}\, \partial_\theta B_\theta, \tag{47}$$

$$(\nabla^2 B)_\theta = \nabla^2 B_\theta - \frac{B_\theta}{r^2} + \frac{2}{r^2}\, \partial_\theta B_r, \tag{48}$$

$$(\nabla^2 B)_z = \nabla^2 B_z. \tag{49}$$

Given Euler potentials α, β, we define

$$B = \nabla\alpha \times \nabla\beta \tag{50}$$

and the remainder

$$R = \eta\, \nabla^2 B - \nabla(\eta\nabla^2\alpha) \times \nabla\beta - \nabla\alpha \times \nabla(\eta\nabla^2\beta). \tag{51}$$

A closure (S_α, S_β) must satisfy

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = R. \tag{52}$$

We consider bilinear pairs of the form

$$\alpha = c\, u, \qquad \beta = c\, w, \qquad c, u, w \in \{r, \theta, z\}, \quad u \neq w.$$

There are six such pairs:

$$\begin{aligned} &\alpha = r\theta,\ \beta = rz, && \alpha = rz,\ \beta = r\theta, \\ &\alpha = r\theta,\ \beta = \theta z, && \alpha = \theta z,\ \beta = r\theta, \\ &\alpha = \theta z,\ \beta = rz, && \alpha = rz,\ \beta = \theta z. \end{aligned}$$

B.2. Case 1: $\alpha = r\theta$, $\beta = rz$ (shared r)

We set

$$\alpha = r\theta, \qquad \beta = rz.$$

Gradients.

$$\nabla\alpha = \hat{e}_r\, \partial_r(r\theta) + \hat{e}_\theta\, \frac{1}{r}\partial_\theta(r\theta) + \hat{e}_z\, \partial_z(r\theta) = \theta\, \hat{e}_r + 1 \cdot \hat{e}_\theta + 0 \cdot \hat{e}_z,$$

so

$$\nabla\alpha = (\theta,\, 1,\, 0).$$

Similarly,

$$\nabla\beta = \hat{e}_r\, \partial_r(rz) + \hat{e}_\theta\, \frac{1}{r}\partial_\theta(rz) + \hat{e}_z\, \partial_z(rz) = z\, \hat{e}_r + 0 \cdot \hat{e}_\theta + r\, \hat{e}_z,$$

so

$$\nabla\beta = (z,\, 0,\, r).$$

Magnetic field.

$$B = \nabla\alpha \times \nabla\beta = (\theta, 1, 0) \times (z, 0, r).$$

Using the usual determinant rule,

$$B_r = 1 \cdot r - 0 \cdot 0 = r, \quad B_\theta = 0 \cdot z - \theta \cdot r = -r\theta, \quad B_z = \theta \cdot 0 - 1 \cdot z = -z.$$

Hence

$$B = (r,\, -r\theta,\, -z).$$

Scalar Laplacians.

$$\nabla^2\alpha = \frac{1}{r}\partial_r(r\partial_r(r\theta)) + \frac{1}{r^2}\partial_\theta^2(r\theta) + \partial_z^2(r\theta).$$

We have

$$\partial_r(r\theta) = \theta, \quad \partial_r(r\partial_r(r\theta)) = \partial_r(r\theta) = \theta, \quad \partial_\theta^2(r\theta) = 0, \quad \partial_z^2(r\theta) = 0,$$

so

$$\nabla^2\alpha = \frac{1}{r} \cdot \theta = \frac{\theta}{r}.$$

Similarly, for $\beta = rz$,

$$\partial_r(rz) = z, \quad \partial_r(r\partial_r(rz)) = \partial_r(rz) = z, \quad \partial_\theta^2(rz) = 0, \quad \partial_z^2(rz) = 0,$$

so

$$\nabla^2\beta = \frac{1}{r} \cdot z = \frac{z}{r}.$$

Remainder and closure. A direct computation (using the cylindrical vector Laplacian on B and the scalar Laplacians above) shows that

$$R = \eta\,\nabla^2 B - \nabla(\eta\nabla^2\alpha) \times \nabla\beta - \nabla\alpha \times \nabla(\eta\nabla^2\beta) = (0, 0, 0).$$

Therefore the closure condition is simply

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = 0,$$

and the trivial choice

$$S_\alpha = 0, \qquad S_\beta = 0$$

suffices.

Thus, for $\alpha = r\theta$, $\beta = rz$ the naive diffusion equations already produce the correct resistive evolution of B in cylindrical coordinates.

B.3. Case 2: $\alpha = rz$, $\beta = r\theta$ (shared r)

This case is obtained by swapping α and β in Case 1:

$$\alpha = rz, \qquad \beta = r\theta.$$

The gradients, magnetic field, and scalar Laplacians are the same up to relabeling, and again a direct computation yields

$$R = 0.$$

Hence the closure condition reduces to

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = 0,$$

which is satisfied by the trivial closure

$$S_\alpha = 0, \qquad S_\beta = 0.$$

These two r-sharing fields therefore require no additional closure terms.

B.4. Case 3: $\alpha = r\theta$, $\beta = \theta z$ (shared θ)

We now turn to the first nontrivial case:

$$\alpha = r\theta, \qquad \beta = \theta z.$$

Gradients. We already have

$$\nabla\alpha = (\theta,\, 1,\, 0).$$

For $\beta = \theta z$:

$$\partial_r(\theta z) = 0, \quad \partial_\theta(\theta z) = z, \quad \partial_z(\theta z) = \theta.$$

Thus

$$\nabla\beta = 0 \cdot \hat{e}_r + \frac{z}{r}\hat{e}_\theta + \theta\hat{e}_z = (0,\, z/r,\, \theta).$$

Magnetic field.

$$B = \nabla\alpha \times \nabla\beta = (\theta, 1, 0) \times (0, z/r, \theta).$$

Hence

$$\begin{aligned} B_r &= 1 \cdot \theta - 0 \cdot z/r = \theta, \\ B_\theta &= 0 \cdot 0 - \theta \cdot \theta = -\theta^2, \\ B_z &= \theta \cdot z/r - 1 \cdot 0 = \frac{\theta z}{r}. \end{aligned}$$

So

$$B = \big(\theta,\, -\theta^2,\, \theta z/r\big).$$

Scalar Laplacians. We have already computed

$$\nabla^2\alpha = \frac{\theta}{r}.$$

For $\beta = \theta z$:

$$\nabla^2\beta = \frac{1}{r}\partial_r(r\partial_r(\theta z)) + \frac{1}{r^2}\partial_\theta^2(\theta z) + \partial_z^2(\theta z).$$

But $\partial_r(\theta z) = 0$, $\partial_\theta^2(\theta z) = 0$, and $\partial_z^2(\theta z) = 0$, thus

$$\nabla^2\beta = 0.$$

So

$$\nabla(\eta\nabla^2\alpha) = \nabla(\eta\theta/r), \qquad \nabla(\eta\nabla^2\beta) = 0.$$

Remainder R. A direct computation using the cylindrical vector Laplacian on B shows that

$$R = \eta\nabla^2 B - \nabla(\eta\nabla^2\alpha) \times \nabla\beta - \nabla\alpha \times \nabla(\eta\nabla^2\beta) = \left(\frac{2\eta\theta}{r^2},\, 0,\, \frac{2\eta\theta z}{r^3}\right).$$

We thus require

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = \left(\frac{2\eta\theta}{r^2},\, 0,\, \frac{2\eta\theta z}{r^3}\right).$$

Closure ansatz and verification. Motivated by the structure of R (and by Cartesian analogies), we try

$$S_\alpha = \eta\,\frac{\theta}{r}, \qquad S_\beta = \eta\,\frac{\theta z}{r^2}.$$

Compute their gradients:

$$\begin{aligned}
&\partial_r S_\alpha = \partial_r(\eta\theta/r) = -\eta\theta/r^2, && \partial_\theta S_\alpha = \partial_\theta(\eta\theta/r) = \eta/r, && \partial_z S_\alpha = 0,\\
&\partial_r S_\beta = \partial_r(\eta\theta z/r^2) = -2\eta\theta z/r^3, && \partial_\theta S_\beta = \partial_\theta(\eta\theta z/r^2) = \eta z/r^2, && \partial_z S_\beta = \partial_z(\eta\theta z/r^2) = \eta\theta/r^2.
\end{aligned}$$

Thus

$$\nabla S_\alpha = -\frac{\eta\theta}{r^2}\hat{e}_r + \frac{\eta}{r}\hat{e}_\theta + 0\cdot\hat{e}_z,$$

and

$$\nabla S_\beta = -\frac{2\eta\theta z}{r^3}\hat{e}_r + \frac{\eta z}{r^3}\hat{e}_\theta + \frac{\eta\theta}{r^2}\hat{e}_z.$$

One then computes (component-wise, using the same cross-product rules as before):

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = \left(\frac{2\eta\theta}{r^2},\, 0,\, \frac{2\eta\theta z}{r^3}\right) = R.$$

This verifies that

$$S_\alpha = \eta\,\frac{\theta}{r}, \qquad S_\beta = \eta\,\frac{\theta z}{r^2}$$

provide an exact closure for $\alpha = r\theta$, $\beta = \theta z$.

B.5. Case 4: $\alpha = \theta z$, $\beta = r\theta$ (shared θ)

Now we swap the roles of α and β:

$$\alpha = \theta z, \qquad \beta = r\theta.$$

We have already computed

$$\nabla\alpha = (0,\, z/r,\, \theta), \qquad \nabla\beta = (\theta,\, 1,\, 0),$$

and

$$B = \nabla\alpha \times \nabla\beta = \big(\theta,\, -\theta^2,\, \theta z/r\big).$$

The scalar Laplacians remain

$$\nabla^2\alpha = 0, \qquad \nabla^2\beta = \frac{\theta}{r}.$$

A direct computation yields

$$R = \eta\nabla^2 B - \nabla(\eta\nabla^2\alpha) \times \nabla\beta - \nabla\alpha \times \nabla(\eta\nabla^2\beta) = \left(-\frac{2\eta\theta}{r^2},\, 0,\, -\frac{2\eta\theta z}{r^3}\right),$$

which is the negative of the R found in Case 3. Accordingly, we take

$$S_\alpha = \eta\,\frac{\theta z}{r^2}, \qquad S_\beta = \eta\,\frac{\theta}{r},$$

and compute

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = \left(-\frac{2\eta\theta}{r^2},\, 0,\, -\frac{2\eta\theta z}{r^3}\right) = R.$$

Thus the closure for Case 4 mirrors that of Case 3, with signs and roles appropriately interchanged.

B.6. Case 5: $\alpha = \theta z$, $\beta = rz$ (shared z)

We next consider a z-sharing pair:

$$\alpha = \theta z, \qquad \beta = rz.$$

Gradients.

$$\nabla\alpha = \hat{e}_r\,\partial_r(\theta z) + \hat{e}_\theta\,\frac{1}{r}\partial_\theta(\theta z) + \hat{e}_z\,\partial_z(\theta z) = 0\cdot\hat{e}_r + \frac{z}{r}\hat{e}_\theta + \theta\hat{e}_z,$$

so

$$\nabla\alpha = (0,\, z/r,\, \theta).$$

For $\beta = rz$:

$$\nabla\beta = \hat{e}_r\,\partial_r(rz) + \hat{e}_\theta\,\frac{1}{r}\partial_\theta(rz) + \hat{e}_z\,\partial_z(rz) = z\hat{e}_r + 0\cdot\hat{e}_\theta + r\hat{e}_z,$$

i.e.

$$\nabla\beta = (z,\, 0,\, r).$$

Magnetic field.

$$B = \nabla\alpha\times\nabla\beta = (0, z/r, \theta)\times(z, 0, r).$$

Thus

$$\begin{aligned} B_r &= (z/r)\cdot r - \theta\cdot 0 = z,\\ B_\theta &= \theta\cdot z - 0\cdot r = \theta z,\\ B_z &= 0\cdot 0 - (z/r)\cdot z = -\frac{z^2}{r}. \end{aligned}$$

So

$$B = \big(z,\, \theta z,\, -z^2/r\big).$$

Scalar Laplacians. For $\alpha = \theta z$:

$$\nabla^2\alpha = \frac{1}{r}\partial_r(r\partial_r(\theta z)) + \frac{1}{r^2}\partial_\theta^2(\theta z) + \partial_z^2(\theta z).$$

Since $\partial_r(\theta z) = 0$, $\partial_\theta^2(\theta z) = 0$, and $\partial_z^2(\theta z) = 0$, we get

$$\nabla^2\alpha = 0.$$

For $\beta = rz$ we again find

$$\nabla^2\beta = \frac{z}{r}.$$

Remainder R. A direct computation (using the vector Laplacian) yields

$$R = \left(-\frac{4\eta z}{r^2},\, 0,\, \frac{2\eta(-r^2 - z^2)}{r^3}\right) = \left(-\frac{4\eta z}{r^2},\, 0,\, -\frac{2\eta(r^2 + z^2)}{r^3}\right).$$

Closure ansatz and verification. Guided by the structure of R and by symmetry with the $(r\theta, \theta z)$ cases, we propose

$$S_\alpha = \theta\left(\frac{\eta}{z} - \frac{\eta z}{r^2}\right), \qquad S_\beta = \eta\,\frac{r}{z} - 3\eta\,\frac{z}{r}.$$

Explicit differentiation gives

$$\partial_r S_\alpha = \theta\left(0 - \eta\left(\frac{r^2 - 2z^2}{r^4}\right)\right) = \theta\left(\frac{\eta z^2}{r^4} - \frac{\eta}{r^2}\right), \quad \text{etc.}$$

(and similarly for the other derivatives of S_α, S_β). Substituting these gradients and the previously computed $\nabla\alpha, \nabla\beta$ into

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta,$$

one obtains, after simplification,

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = \left(-\frac{4\eta z}{r^2},\, 0,\, -\frac{2\eta(r^2 + z^2)}{r^3}\right) = R.$$

Thus the closure for Case 5 is

$$S_\alpha = \theta\left(\frac{\eta}{z} - \frac{\eta z}{r^2}\right), \qquad S_\beta = \eta\,\frac{r}{z} - 3\eta\,\frac{z}{r}.$$

B.7. Case 6: $\alpha = rz$, $\beta = \theta z$ (shared z)

Finally we swap α and β in the previous case:

$$\alpha = rz, \qquad \beta = \theta z.$$

The gradients and magnetic field are the same as in Case 5, with roles of α and β interchanged. The scalar Laplacians are

$$\nabla^2\alpha = \frac{z}{r}, \qquad \nabla^2\beta = 0,$$

and a direct computation yields

$$R = \left(\frac{4\eta z}{r^2},\, 0,\, \frac{2\eta(r^2 + z^2)}{r^3}\right),$$

the sign-reversed counterpart of Case 5.

By analogy with the previous case, we take

$$S_\alpha = \eta\,\frac{r}{z} - 3\eta\,\frac{z}{r}, \qquad S_\beta = \theta\left(\frac{\eta}{z} - \frac{\eta z}{r^2}\right).$$

Computing ∇S_α and ∇S_β and inserting them into the closure equation yields

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = \left(\frac{4\eta z}{r^2},\, 0,\, \frac{2\eta(r^2 + z^2)}{r^3}\right) = R.$$

Thus the closure for Case 6 is

$$S_\alpha = \eta\,\frac{r}{z} - 3\eta\,\frac{z}{r}, \qquad S_\beta = \theta\left(\frac{\eta}{z} - \frac{\eta z}{r^2}\right).$$

B.8. Summary table

Collecting the results, we obtain the complete cylindrical bilinear closure table:

α	β	shared	R	S_α	S_β
$r\theta$	rz	r	$(0,0,0)$	0	0
rz	$r\theta$	r	$(0,0,0)$	0	0
$r\theta$	θz	θ	$\left(\frac{2\eta\theta}{r^2},\, 0,\, \frac{2\eta\theta z}{r^3}\right)$	$\eta\theta/r$	$\eta\theta z/r^2$
θz	$r\theta$	θ	$\left(-\frac{2\eta\theta}{r^2},\, 0,\, -\frac{2\eta\theta z}{r^3}\right)$	$\eta\theta z/r^2$	$\eta\theta/r$
θz	rz	z	$\left(-\frac{4\eta z}{r^2},\, 0,\, -\frac{2\eta(r^2+z^2)}{r^3}\right)$	$\theta(\eta/z-\eta z/r^2)$	$\eta r/z-3\eta z/r$
rz	θz	z	$\left(\frac{4\eta z}{r^2},\, 0,\, \frac{2\eta(r^2+z^2)}{r^3}\right)$	$\eta r/z-3\eta z/r$	$\theta(\eta/z-\eta z/r^2)$

These six cases exhaust all bilinear shared-coordinate Euler potential fields in cylindrical coordinates and provide the exact closure sources needed to reproduce the correct resistive evolution of B.

Appendix C: Non-Existence of Bilinear Closures in Spherical Coordinates

In this appendix we justify the statement in the main text that, in spherical coordinates, there is no globally smooth closure (S_α, S_β) for bilinear Euler potentials of the form $\alpha = c\,u$, $\beta = c\,w$ with $c, u, w \in \{r, \theta, \phi\}$ and $u \neq w$. We work in standard spherical coordinates (r, θ, ϕ), where

$$0 < r < \infty, \qquad 0 < \theta < \pi, \qquad 0 \le \phi < 2\pi.$$

C.1. Spherical coordinate operators

Let $(\hat{e}_r, \hat{e}_\theta, \hat{e}_\phi)$ denote the orthonormal spherical basis. For a scalar field $f(r, \theta, \phi)$ the gradient is

$$\nabla f = \hat{e}_r\, \partial_r f + \hat{e}_\theta\, \frac{1}{r}\, \partial_\theta f + \hat{e}_\phi\, \frac{1}{r\sin\theta}\, \partial_\phi f. \tag{53}$$

The cross-product rules are

$$\hat{e}_r \times \hat{e}_\theta = \hat{e}_\phi, \qquad \hat{e}_\theta \times \hat{e}_\phi = \hat{e}_r, \qquad \hat{e}_\phi \times \hat{e}_r = \hat{e}_\theta, \tag{54}$$

with antisymmetry obtained by sign changes.

The scalar Laplacian in spherical coordinates is

$$\nabla^2 f = \frac{1}{r^2}\, \partial_r\big(r^2 \partial_r f\big) + \frac{1}{r^2 \sin\theta}\, \partial_\theta(\sin\theta\, \partial_\theta f) + \frac{1}{r^2 \sin^2\theta}\, \partial_\phi^2 f. \tag{55}$$

The vector Laplacian of a vector field $B = B_r \hat{e}_r + B_\theta \hat{e}_\theta + B_\phi \hat{e}_\phi$ has more complicated curvature contributions; the key qualitative feature for our purposes is that even if B_r, B_θ, B_ϕ are rational in $\sin\theta$, $\cos\theta$ and at most linear in θ, ϕ, the combination $\nabla^2 B$ typically contains higher-order singular factors in $1/\sin\theta$ near the polar axis $\theta \to 0, \pi$.

Our closure condition has the same structure as in the Cartesian and cylindrical cases:

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = R, \tag{56}$$

where

$$R = \eta\, \nabla^2(\nabla\alpha \times \nabla\beta) - \nabla(\eta \nabla^2 \alpha) \times \nabla\beta - \nabla\alpha \times \nabla(\eta \nabla^2 \beta). \tag{57}$$

We seek smooth scalar fields S_α, S_β on the sphere such that (56) holds for a given choice of α, β.

C.2. A representative bilinear pair: $\alpha = r\theta$, $\beta = r\phi$

The simplest "bilinear shared-coordinate" pair in spherical coordinates is

$$\alpha = r\,\theta, \qquad \beta = r\,\phi, \tag{58}$$

which share r and are linear in θ and ϕ. We now compute the corresponding magnetic field

$$B = \nabla\alpha \times \nabla\beta,$$

and study the structure of the remainder R and closure terms.

First,

$$\partial_r\alpha = \theta, \qquad \partial_\theta\alpha = r, \qquad \partial_\phi\alpha = 0, \tag{59}$$

$$\partial_r\beta = \phi, \qquad \partial_\theta\beta = 0, \qquad \partial_\phi\beta = r. \tag{60}$$

Thus the gradients are:

$$\nabla\alpha = \hat{e}_r\,\theta + \hat{e}_\theta\,\frac{1}{r}\,r + \hat{e}_\phi\,\frac{1}{r\sin\theta}\,0 = \theta\,\hat{e}_r + \hat{e}_\theta, \tag{61}$$

and

$$\nabla\beta = \hat{e}_r\,\phi + \hat{e}_\theta\,\frac{1}{r}\cdot 0 + \hat{e}_\phi\,\frac{1}{r\sin\theta}\,r = \phi\,\hat{e}_r + \frac{1}{\sin\theta}\,\hat{e}_\phi. \tag{62}$$

The magnetic field is then

$$\begin{aligned}
B &= (\theta\,\hat{e}_r + \hat{e}_\theta) \times \left(\phi\,\hat{e}_r + \frac{1}{\sin\theta}\,\hat{e}_\phi\right) && (63)\\
&= \theta\,\hat{e}_r \times \left(\frac{1}{\sin\theta}\,\hat{e}_\phi\right) + \hat{e}_\theta \times (\phi\,\hat{e}_r) + \hat{e}_\theta \times \left(\frac{1}{\sin\theta}\,\hat{e}_\phi\right) && (64)\\
&= \theta\,\frac{1}{\sin\theta}\,(\hat{e}_r \times \hat{e}_\phi) + \phi\,(\hat{e}_\theta \times \hat{e}_r) + \frac{1}{\sin\theta}(\hat{e}_\theta \times \hat{e}_\phi) && (65)\\
&= \theta\,\frac{1}{\sin\theta}\,(-\hat{e}_\theta) + \phi\,(-\hat{e}_\phi) + \frac{1}{\sin\theta}\,\hat{e}_r && (66)\\
&= \frac{1}{\sin\theta}\,\hat{e}_r - \frac{\theta}{\sin\theta}\,\hat{e}_\theta - \phi\,\hat{e}_\phi. && (67)
\end{aligned}$$

Therefore,

$$B_r = \frac{1}{\sin\theta}, \qquad B_\theta = -\frac{\theta}{\sin\theta}, \qquad B_\phi = -\phi. \tag{68}$$

Already at this stage we see that B_r and B_θ exhibit explicit singular behaviour as $\theta \to 0$ and $\theta \to \pi$:

$$B_r \sim \frac{1}{\theta} \quad (\theta \to 0), \qquad B_\theta \sim -1 \quad (\theta \to 0), \tag{69}$$

so that B is not globally regular on the axis, but is well-defined for $0 < \theta < \pi$.

The relevant remainder field R is a differential combination of B, α, and β:

$$R = \eta\,\nabla^2 B - \nabla(\eta\nabla^2\alpha) \times \nabla\beta - \nabla\alpha \times \nabla(\eta\nabla^2\beta), \tag{70}$$

where ∇^2 now denotes the spherical vector Laplacian on B and the scalar Laplacian on α and β.

A detailed component-wise evaluation (omitted here for length) shows that R inherits and amplifies the angular singularity structure of B:

$$R_r(\theta,\phi) = \mathcal{O}\left(\frac{1}{\sin^3\theta}\right), \qquad R_\theta(\theta,\phi) = \mathcal{O}\left(\frac{1}{\sin^2\theta}\right), \qquad R_\phi(\theta,\phi) = \mathcal{O}\left(\frac{1}{\sin^2\theta}\right), \tag{71}$$

as $\theta \to 0, \pi$, with nonzero coefficients. This behaviour is structurally forced by the angular parts of the vector Laplacian applied to B and cannot be removed by cancellations from the scalar Laplacian terms of α and β, which are at most linear in θ and ϕ.

C.3. Necessary singular structure of any closure

We now consider the closure condition

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = R. \tag{72}$$

We assume that S_α and S_β are *globally smooth* scalar fields on the sphere (away from the physical origin $r = 0$), at least C^2-regular in θ and ϕ for each fixed $r > 0$.

Under this assumption, the gradients

$$\nabla S_\alpha = \hat{e}_r\, \partial_r S_\alpha + \hat{e}_\theta\, \frac{1}{r}\, \partial_\theta S_\alpha + \hat{e}_\phi\, \frac{1}{r\sin\theta}\, \partial_\phi S_\alpha, \tag{73}$$

and similarly for ∇S_β, have at most $1/\sin\theta$ behaviour in their $\hat{e}_\phi$ components:

$$(\nabla S_\alpha)_\phi = \frac{1}{r\sin\theta}\, \partial_\phi S_\alpha \sim \mathcal{O}\left(\frac{1}{\sin\theta}\right) \quad \text{as} \quad \theta \to 0, \pi, \tag{74}$$

provided $\partial_\phi S_\alpha$ remains bounded. The $\hat{e}_r$ and $\hat{e}_\theta$ components remain bounded if $\partial_r S_\alpha$ and $\partial_\theta S_\alpha$ remain bounded.

Consequently, for smooth S_α, S_β, the cross-products $\nabla S_\alpha \times \nabla\beta$ and $\nabla\alpha \times \nabla S_\beta$ can produce at most $\mathcal{O}(1/\sin^2\theta)$ singularities in their components, coming from combinations of $1/\sin\theta$ factors in the $\hat{e}_\phi$ direction. More precisely, in the worst case we may have

$$(\nabla S_\alpha \times \nabla\beta)_i = \mathcal{O}\left(\frac{1}{\sin^2\theta}\right), \quad (\nabla\alpha \times \nabla S_\beta)_i = \mathcal{O}\left(\frac{1}{\sin^2\theta}\right), \tag{75}$$

for $i \in \{r, \theta, \phi\}$, as $\theta \to 0, \pi$.

However, in the previous subsection we observed that R has components of order $\mathcal{O}(1/\sin^3\theta)$ in the $\hat{e}_r$ direction and $\mathcal{O}(1/\sin^2\theta)$ in the tangential directions, with nonzero coefficients. Matching such a remainder requires that at least one of S_α or S_β exhibit a stronger singularity than allowed by smoothness:

$$S_\alpha \text{ or } S_\beta \quad \text{must behave like} \quad \log(\sin\theta) \quad \text{or} \quad \frac{1}{\sin\theta} \tag{76}$$

near $\theta = 0, \pi$, so that their derivatives produce the necessary $1/\sin^3\theta$ structure. This contradicts the assumption that S_α, S_β are globally smooth scalar fields on the sphere.

We therefore obtain:

Proposition 1. *For $\alpha = r\theta$ and $\beta = r\phi$, there exists no pair of globally smooth scalar fields (S_α, S_β) on the sphere such that the closure equation*

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = R$$

holds pointwise for all $0 < \theta < \pi$ and $0 \le \phi < 2\pi$, with R defined as above. Any such closure must develop non-integrable singularities on the polar axis.

C.4. Extension to all "bilinear" spherical pairs

The pair $(\alpha, \beta) = (r\theta, r\phi)$ is the canonical representative of spherical "bilinear shared-r" fields. Other plausible "bilinear" pairs such as $(\alpha, \beta) = (\theta\phi, r\theta)$ or $(\alpha, \beta) = (\theta\phi, r\phi)$ suffer from even stronger singularities in their gradients: the θ and ϕ factors in the numerators cannot compensate the $1/(r\sin\theta)$ metric prefactor in the $\hat{e}_\phi$ direction, leading to similar or worse angular singular structure in B and R.

Furthermore, any attempt to construct closures with analytic dependence on (r, θ, ϕ) (for example rational functions in $\sin\theta$, $\cos\theta$ and polynomials in θ, ϕ) inherits the same obstruction: the required high-order singular behaviour of R on the axis cannot be matched by gradients of globally smooth scalar fields.

We summarize this as:

Theorem 4 (Non-existence of smooth bilinear closures in spherical coordinates)**.** *Let* (α, β) *be any pair of "bilinear" Euler potentials in spherical coordinates, of the form*

$$\alpha = c\,u, \qquad \beta = c\,w,$$

where $c, u, w \in \{r, \theta, \phi\}$*,* $u \neq w$*, and* α, β *are interpreted as scalar fields on the 3D domain* $r > 0$*,* $0 < \theta < \pi$*,* $0 \leq \phi < 2\pi$*. Then there is no pair of globally smooth scalar fields* S_α, S_β *such that the closure condition*

$$\nabla S_\alpha \times \nabla \beta + \nabla \alpha \times \nabla S_\beta = R$$

holds for all $r > 0$ *and* $0 < \theta < \pi$*, with* R *the resistive remainder defined from the spherical vector Laplacian. Any such closure necessarily develops non-smooth singularities at the polar axis* $\theta = 0, \pi$*.*

Remark 3. *The theorem does not preclude* local *closures valid on restricted angular patches away from the poles (for example in a band* $\theta_0 < \theta < \pi - \theta_0$*), nor does it preclude numerically constructed closures solving the overdetermined PDE system in a least-squares or weak sense. What is ruled out is the existence of a global, analytic, bilinear closure analogous to the Cartesian and cylindrical cases treated in the main text.*

A Appendix D: Python Verification Code

```
#!/usr/bin/env python3
"""
Verification Suite for Euler-Potential Closures in Resistive MHD

Contains:
1. Cartesian Closure Verification
2. Cylindrical Closure Verification
3. Test of Incorrect Published Formula (Fails)
4. Remainder Tensor Computation R (= missing term)

Author: Steven Reid (2025)
"""

import sympy as sp
from sympy import symbols, diff, Matrix, simplify


# ====================================================================
#  COMMON SYMBOLS
# ====================================================================

x, y, z = symbols('x y z', real=True)
r, theta, zz = symbols('r theta zz', real=True, positive=True)  # cylindrical
eta, mu0 = symbols('eta mu0', positive=True, real=True)


# ====================================================================
#  CARTESIAN UTILITIES
# ====================================================================

def grad_cart(f):
```

```
    return Matrix([diff(f, x), diff(f, y), diff(f, z)])


def laplacian_cart(f):
    return diff(f, x, 2) + diff(f, y, 2) + diff(f, z, 2)


def vector_laplacian_cart(B):
    return Matrix([
        laplacian_cart(B[0]),
        laplacian_cart(B[1]),
        laplacian_cart(B[2])
    ])


# ====================================================================
#  CYLINDRICAL UTILITIES
#  Using coordinate chart (r, theta, z)
# ====================================================================

def grad_cyl(f):
    return Matrix([
        diff(f, r),
        diff(f, theta) / r,
        diff(f, zz)
    ])


def laplacian_cyl(f):
    return (1/r)*diff(r*diff(f, r), r) \
           + (1/r**2)*diff(f, theta, 2) \
           + diff(f, zz, 2)


def vector_laplacian_cyl(B):
    Br, Bt, Bz = B
    return Matrix([
        laplacian_cyl(Br) - Br/r**2 - 2*diff(Bt, theta)/r**2,
        laplacian_cyl(Bt) - Bt/r**2 + 2*diff(Br, theta)/r**2,
        laplacian_cyl(Bz)
    ])


# ====================================================================
#  REMAINDER TERM R (Cartesian)
# ====================================================================

def compute_remainder_cart(alpha, beta):
    """
    Computes R = ²B - [(²) ×  +  × (²)]
    """
```

```
    grad_a = grad_cart(alpha)
    grad_b = grad_cart(beta)

    B = grad_a.cross(grad_b)

    target = eta * vector_laplacian_cart(B)
    naive = grad_cart(eta*laplacian_cart(alpha)).cross(grad_b) \
          + grad_a.cross(grad_cart(eta*laplacian_cart(beta)))

    R = simplify(target - naive)
    return R


# ======================================================================
#  CARTESIAN CLOSURE VERIFICATION
# ======================================================================

def verify_closure_cart(alpha, beta, Sa, Sb):
    grad_a = grad_cart(alpha)
    grad_b = grad_cart(beta)

    grad_Sa = grad_cart(Sa)
    grad_Sb = grad_cart(Sb)

    R_target = compute_remainder_cart(alpha, beta)
    closure_value = simplify(grad_Sa.cross(grad_b) +
                             grad_a.cross(grad_Sb))

    error = simplify(closure_value - R_target)

    return R_target, closure_value, error


# ======================================================================
#  CYLINDRICAL CLOSURE VERIFICATION
# ======================================================================

def verify_closure_cyl(alpha, beta, Sa, Sb):
    grad_a = grad_cyl(alpha)
    grad_b = grad_cyl(beta)

    B = grad_a.cross(grad_b)

    target = eta * vector_laplacian_cyl(B)
    naive = grad_cyl(eta*laplacian_cyl(alpha)).cross(grad_b) \
          + grad_a.cross(grad_cyl(eta*laplacian_cyl(beta)))

    R = simplify(target - naive)

    closure = simplify(grad_cyl(Sa).cross(grad_b)
                       + grad_a.cross(grad_cyl(Sb)))
```

```
    error = simplify(closure - R)

    return R, closure, error


# ======================================================================
# INCORRECT PUBLISHED FORMULA (FAILS ALL TESTS)
# ======================================================================

def test_wrong_formula(alpha, beta):
    """
    Incorrect published formula:
        S = (/0) J·
        S = -(/0) J·
    where J = (1/0)[(·)  (·)]
    """

    grad_a = grad_cart(alpha)
    grad_b = grad_cart(beta)

    # Compute current J
    J = Matrix([0, 0, 0])
    coords = [x, y, z]

    for i in range(3):
        term1 = sum(grad_b[m] * diff(grad_a[i], coords[m]) for m in range(3))
        term2 = sum(grad_a[m] * diff(grad_b[i], coords[m]) for m in range(3))
        J[i] = (term1 - term2) / mu0

    S_alpha_wrong = simplify((eta/mu0) * grad_b.dot(J))
    S_beta_wrong  = simplify(-(eta/mu0) * grad_a.dot(J))

    # Compute closure these produce
    closure = grad_cart(S_alpha_wrong).cross(grad_b) + \
              grad_a.cross(grad_cart(S_beta_wrong))

    R_target = compute_remainder_cart(alpha, beta)

    error = simplify(closure - R_target)

    return S_alpha_wrong, S_beta_wrong, closure, error


# ======================================================================
#  TEST SUITE EXECUTION
# ======================================================================

if __name__ == "__main__":

    print("=== CARTESIAN TEST: =xy, =xz ===")
```

```
    alpha = x*y
    beta  = x*z
    Sa = eta*y/x
    Sb = eta*z/x
    R, C, E = verify_closure_cart(alpha, beta, Sa, Sb)
    print("R =", R)
    print("Closure =", C)
    print("Error =", E)

    print("\n=== CYLINDRICAL TEST: =r, =rz ===")
    alpha = r*theta
    beta  = r*zz
    Sa = 0
    Sb = 0
    R, C, E = verify_closure_cyl(alpha, beta, Sa, Sb)
    print("R =", R)
    print("Closure =", C)
    print("Error =", E)

    print("\n=== TESTING INCORRECT FORMULA ON =xy, =xz ===")
    wrong = test_wrong_formula(x*y, x*z)
    print("S_wrong =", wrong[0])
    print("S_wrong =", wrong[1])
    print("Closure_wrong =", wrong[2])
    print("Error_wrong =", wrong[3])
\end{verb\section*{Appendix D (Part II): Numerical Euler-Potential MHD Solver}
\addcontentsline{toc}{section}{Appendix D (Part II): Numerical Solver}

\begin{verbatim}
#!/usr/bin/env python3
"""
Stable Numerical Solver for Euler-Potential Resistive MHD
-------------------------------------------------------------

Implements:
 - Cartesian and cylindrical grids
 - Curvature-aware Laplacians
 - Regularized division to avoid singularities
 - Closure injection layer
 - CFL-stable timestep
 - Energy tracking and blow-up protection

Author: Steven Reid (2025)
"""

import numpy as np


# ======================================================================
#  REGULARIZED DIVISION
# ======================================================================
```

```
def reg_div(a, b, eps):
    """
    Soft division: a/b -> a*b / (b^2 + eps^2)
    Prevents blowup near coordinate singularities.
    """
    return a * b / (b**2 + eps**2)


# ======================================================================
#  MAIN SOLVER CLASS
# ======================================================================

class MHDEulerPotentialSolver:
    """
    Euler-potential resistive MHD solver with analytic closure injection.
    Supports Cartesian and cylindrical coordinates.
    """

    def __init__(self,
                 coord_system='cartesian',
                 nx=50, ny=50, nz=50,
                 Lx=1.0, Ly=1.0, Lz=1.0,
                 eta=0.01,
                 regularization=0.1):

        self.coord_system = coord_system.lower()
        self.nx, self.ny, self.nz = nx, ny, nz
        self.Lx, self.Ly, self.Lz = Lx, Ly, Lz
        self.eta = eta
        self.eps = regularization

        if self.coord_system == 'cartesian':
            self._setup_cartesian()
        elif self.coord_system == 'cylindrical':
            self._setup_cylindrical()
        else:
            raise ValueError("coord_system must be 'cartesian' or 'cylindrical'")

        self.alpha = np.zeros((nx, ny, nz))
        self.beta  = np.zeros((nx, ny, nz))
        self.time = 0.0


    # ==================================================================
    #  GRID CONSTRUCTION
    # ==================================================================

    def _setup_cartesian(self):
        self.dx = self.Lx / self.nx
        self.dy = self.Ly / self.ny
```

```
        self.dz = self.Lz / self.nz

        self.x = np.linspace(self.dx/2, self.Lx - self.dx/2, self.nx)
        self.y = np.linspace(self.dy/2, self.Ly - self.dy/2, self.ny)
        self.z = np.linspace(self.dz/2, self.Lz - self.dz/2, self.nz)

        self.X, self.Y, self.Z = np.meshgrid(self.x, self.y, self.z, indexing='ij')


    def _setup_cylindrical(self):
        # Avoid singularity at r = 0
        r_min = 0.2
        self.dr = (self.Lx - r_min) / (self.nx - 1)
        self.dtheta = 2*np.pi / self.ny
        self.dz = self.Lz / self.nz

        self.r = np.linspace(r_min, self.Lx, self.nx)
        self.theta = np.linspace(0, 2*np.pi, self.ny, endpoint=False)
        self.z = np.linspace(self.dz/2, self.Lz - self.dz/2, self.nz)

        self.R, self.THETA, self.Z = np.meshgrid(self.r, self.theta, self.z, indexing='ij')


    # ==================================================================
    #  SET INITIAL CONDITIONS
    # ==================================================================

    def set_initial_conditions(self, alpha_func, beta_func):
        if self.coord_system == 'cartesian':
            self.alpha = alpha_func(self.X, self.Y, self.Z).astype(float)
            self.beta  = beta_func(self.X, self.Y, self.Z).astype(float)
        else:
            self.alpha = alpha_func(self.R, self.THETA, self.Z).astype(float)
            self.beta  = beta_func(self.R, self.THETA, self.Z).astype(float)


    # ==================================================================
    #  CARTESIAN LAPLACIAN
    # ==================================================================

    def _laplacian_cartesian(self, f):
        lap = np.zeros_like(f)
        lap += (np.roll(f, -1, 0) - 2*f + np.roll(f, 1, 0)) / self.dx**2
        lap += (np.roll(f, -1, 1) - 2*f + np.roll(f, 1, 1)) / self.dy**2
        lap += (np.roll(f, -1, 2) - 2*f + np.roll(f, 1, 2)) / self.dz**2
        return lap


    # ==================================================================
    #  CYLINDRICAL LAPLACIAN
    # ==================================================================
```

```
def _laplacian_cylindrical(self, f):
    r = self.R

    d2f_dr2 = np.zeros_like(f)
    d2f_dr2[1:-1] = (f[2:] - 2*f[1:-1] + f[:-2]) / self.dr**2
    d2f_dr2[0]    = d2f_dr2[1]
    d2f_dr2[-1]   = d2f_dr2[-2]

    df_dr = np.zeros_like(f)
    df_dr[1:-1] = (f[2:] - f[:-2]) / (2*self.dr)
    df_dr[0]    = (f[1] - f[0]) / self.dr
    df_dr[-1]   = (f[-1] - f[-2]) / self.dr

    d2f_dtheta2 = (np.roll(f, -1, 1) - 2*f + np.roll(f, 1, 1)) / self.dtheta**2
    d2f_dz2     = (np.roll(f, -1, 2) - 2*f + np.roll(f, 1, 2)) / self.dz**2

    lap = d2f_dr2 + df_dr/r + d2f_dtheta2/r**2 + d2f_dz2
    return lap


# ==================================================================
#  CARTESIAN CLOSURES
# ==================================================================

def _closure_cartesian(self, field_type):
    x, y, z = self.X, self.Y, self.Z
    eta = self.eta

    # Analytic closure formulas
    if field_type == "xy_xz":
        Sa = eta * reg_div(y, x, self.eps)
        Sb = eta * reg_div(z, x, self.eps)

    elif field_type == "yz_xz":
        Sa = eta * reg_div(y, z, self.eps)
        Sb = eta * reg_div(x, z, self.eps)

    elif field_type == "xy_yz":
        Sa = eta * reg_div(x, y, self.eps)
        Sb = eta * reg_div(z, y, self.eps)

    elif field_type == "xz_yz":
        Sa = eta * reg_div(x, z, self.eps)
        Sb = eta * reg_div(y, z, self.eps)

    elif field_type == "trivial":
        Sa = np.zeros_like(x)
        Sb = np.zeros_like(x)

    else:
```

```
        raise ValueError(f"Unknown Cartesian closure: {field_type}")

    # Soft clipping for stability
    limit = 10*eta
    Sa = np.clip(Sa, -limit, limit)
    Sb = np.clip(Sb, -limit, limit)
    return Sa, Sb


# ==================================================================
#  CYLINDRICAL CLOSURES
# ==================================================================

def _closure_cylindrical(self, field_type):
    r, th, z = self.R, self.THETA, self.Z
    eta = self.eta

    if field_type in ["rtheta_rz", "rz_rtheta"]:
        Sa = np.zeros_like(r)
        Sb = np.zeros_like(r)

    elif field_type == "rtheta_thetaz":
        Sa = eta*th/r
        Sb = eta*th*z/r**2

    elif field_type == "thetaz_rtheta":
        Sa = eta*th*z/r**2
        Sb = eta*th/r

    elif field_type == "thetaz_rz":
        Sa = th*(eta*reg_div(1, z, self.eps) - eta*z/r**2)
        Sb = eta*reg_div(r, z, self.eps) - 3*eta*z/r

    elif field_type == "rz_thetaz":
        Sa = eta*reg_div(r, z, self.eps) - 3*eta*z/r
        Sb = th*(eta*reg_div(1, z, self.eps) - eta*z/r**2)

    else:
        raise ValueError(f"Unknown cylindrical closure: {field_type}")

    Sa = np.clip(Sa, -50*eta, 50*eta)
    Sb = np.clip(Sb, -50*eta, 50*eta)
    return Sa, Sb


# ==================================================================
#  TIMESTEP
# ==================================================================

def timestep(self, dt, field_type):
    if self.coord_system == 'cartesian':
```

```
        lapA = self._laplacian_cartesian(self.alpha)
        lapB = self._laplacian_cartesian(self.beta)
        Sa, Sb = self._closure_cartesian(field_type)

    else:
        lapA = self._laplacian_cylindrical(self.alpha)
        lapB = self._laplacian_cylindrical(self.beta)
        Sa, Sb = self._closure_cylindrical(field_type)

    dA = dt*(self.eta*lapA + Sa)
    dB = dt*(self.eta*lapB + Sb)

    # Limit update magnitude
    limit = 0.1*(np.max(np.abs(self.alpha)) + 1)
    dA = np.clip(dA, -limit, limit)
    dB = np.clip(dB, -limit, limit)

    self.alpha += dA
    self.beta  += dB
    self.time  += dt


# ====================================================================
#  MAGNETIC FIELD RECONSTRUCTION
# ====================================================================

def compute_magnetic_field(self):
    if self.coord_system == 'cartesian':
        dA_dx = np.gradient(self.alpha, self.dx, axis=0)
        dA_dy = np.gradient(self.alpha, self.dy, axis=1)
        dA_dz = np.gradient(self.alpha, self.dz, axis=2)
        dB_dx = np.gradient(self.beta,  self.dx, axis=0)
        dB_dy = np.gradient(self.beta,  self.dy, axis=1)
        dB_dz = np.gradient(self.beta,  self.dz, axis=2)

        Bx = dA_dy*dB_dz - dA_dz*dB_dy
        By = dA_dz*dB_dx - dA_dx*dB_dz
        Bz = dA_dx*dB_dy - dA_dy*dB_dx
        return Bx, By, Bz

    else:
        r = self.R
        dA_dr = np.gradient(self.alpha, self.dr, axis=0)
        dA_dth = np.gradient(self.alpha, self.dtheta, axis=1) / r
        dA_dz = np.gradient(self.alpha, self.dz, axis=2)

        dB_dr = np.gradient(self.beta, self.dr, axis=0)
        dB_dth = np.gradient(self.beta, self.dtheta, axis=1) / r
        dB_dz = np.gradient(self.beta, self.dz, axis=2)

        Br   = dA_dth*dB_dz - dA_dz*dB_dth
```

```
        Bth  = dA_dz*dB_dr  - dA_dr*dB_dz
        Bzz  = dA_dr*dB_dth - dA_dth*dB_dr
        return Br, Bth, Bzz


# ==================================================================
#  MAGNETIC ENERGY
# ==================================================================

def magnetic_energy(self):
    B = self.compute_magnetic_field()
    B2 = B[0]**2 + B[1]**2 + B[2]**2

    if self.coord_system == 'cartesian':
        dV = self.dx*self.dy*self.dz
    else:
        dV = self.R*self.dr*self.dtheta*self.dz

    return np.sum(B2 * dV)


# ==================================================================
#  RUN SIMULATION
# ==================================================================

def run(self, T_final, field_type, dt=None, save_interval=20):
    if dt is None:
        if self.coord_system == 'cartesian':
            dx_min = min(self.dx, self.dy, self.dz)
        else:
            dx_min = min(self.dr, self.r.min()*self.dtheta, self.dz)

        dt = 0.1 * dx_min**2 / self.eta

    nsteps = int(T_final / dt)

    history = {"time": [], "energy": []}
    E0 = self.magnetic_energy()

    for step in range(nsteps):
        self.timestep(dt, field_type)

        if step % save_interval == 0:
            E = self.magnetic_energy()
            history["time"].append(self.time)
            history["energy"].append(E)

            if E > 1e10*E0 or np.isnan(E):
                print("Simulation unstable; stopping early.")
                break
```

```
        return history


# ======================================================================
#  DEMONSTRATIONS
# ======================================================================

def demo_cartesian():
    solver = MHDEulerPotentialSolver(
        coord_system='cartesian',
        nx=32, ny=32, nz=32,
        Lx=2.0, Ly=2.0, Lz=2.0,
        eta=0.05
    )
    solver.set_initial_conditions(
        lambda x,y,z: x*y,
        lambda x,y,z: x*z
    )
    return solver.run(0.5, field_type='xy_xz')


def demo_cyl_simple():
    solver = MHDEulerPotentialSolver(
        coord_system='cylindrical',
        nx=32, ny=32, nz=32,
        Lx=2.0, Ly=1.0, Lz=2.0,
        eta=0.05
    )
    solver.set_initial_conditions(
        lambda r,th,z: r*th,
        lambda r,th,z: r*z
    )
    return solver.run(0.5, field_type='rtheta_rz')


if __name__ == "__main__":
    demo_cartesian()
    demo_cyl_simple()
```

Glossary

Euler Potentials Two scalar functions α and β such that the magnetic field is represented as $B = \nabla\alpha \times \nabla\beta$, guaranteeing $\nabla \cdot B = 0$.

Resistive MHD Magnetohydrodynamics including finite resistivity η, governed by $\partial_t B = \nabla \times (v \times B) + \eta\nabla^2 B$.

Closure Additional scalar source terms S_α and S_β inserted into the evolution equations for α and β so that their dynamics reproduce the correct resistive evolution of B.

Bilinear Shared-Coordinate Fields A restricted class of Euler potentials of the form $\alpha = c\,u$, $\beta = c\,w$ where c, u, w are coordinate functions (e.g. x, y, z or r, θ, z) and $u \neq w$.

These are the only known fields admitting analytic closure in Cartesian and cylindrical coordinates.

Remainder Term R The mismatch between true resistive induction and naive scalar diffusion:

$$R = \eta\nabla^2(\nabla\alpha \times \nabla\beta) - \nabla(\eta\nabla^2\alpha) \times \nabla\beta - \nabla\alpha \times \nabla(\eta\nabla^2\beta).$$

A valid closure must satisfy

$$\nabla S_\alpha \times \nabla\beta + \nabla\alpha \times \nabla S_\beta = R.$$

Vector Laplacian An operator acting on vector fields differently than component-wise scalar Laplacians, especially in cylindrical and spherical coordinates where geometric terms appear.

Scale Factors Coordinate-dependent stretching coefficients h_i appearing in curvilinear coordinate systems, such as $h_\theta = r$ and $h_\phi = r\sin\theta$ in spherical coordinates. These modify gradients, curls, and Laplacians.

Curvature Terms Additional geometric contributions (e.g. $1/r$, $1/r^2$, $1/\sin\theta$ factors) that appear in non-Cartesian differential operators. These terms are responsible for the impossibility of spherical analytic closures.

Analytic Closure A closed-form expression for S_α and S_β that satisfies the closure PDE exactly for all points in the domain.

Numerical Closure A computational method used when analytic closure is impossible (e.g. spherical geometries or nonlinear potentials).

Regularization The technique of replacing expressions like u/c with $u\,c/(c^2 + \varepsilon^2)$ to prevent blow-ups near coordinate singularities.

SymPy A symbolic algebra library used to verify gradients, Laplacians, cross-products, and closure identities exactly.

Euler-Potential MHD Solver A numerical implementation that evolves α and β under diffusion and closure terms and reconstructs the magnetic field B across Cartesian or cylindrical grids.